Financial QA Chatbot

By: Jaden Kuriakose, Dev Sheth, Dev Singh, Abhinav Binu, Raisa Reza, & Ahmed Fouad

Objectives

- Allow users to upload financial documents
- Provide insights quickly that are easy to read and understand
- Give users easy access to financial expertise and document QA
- Save users time when having to read long, complex documents

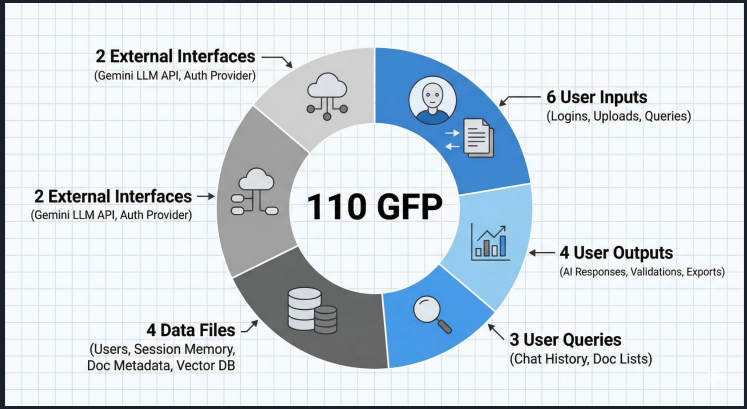
Cost Estimation: Function Point Analysis

Methodology: Utilized Function Point Analysis (FPA) to measure system size based on functionality rather than lines of code.

Component Breakdown:

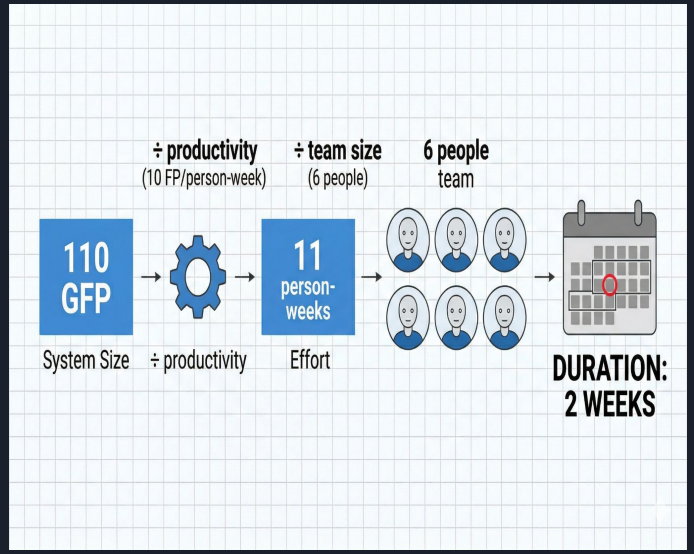
- **User Inputs:** 6 (Logins, uploads, queries)
- **User Outputs:** 4 (AI responses, validations, exports)
- **User Queries:** 3 (Chat history, document lists)
- **Data Files:** 4 (Users, session memory, document metadata, Vector DB)
- **External Interfaces:** 2 (Gemini LLM API, Auth provider)

Complexity Assessment: Rated "Average" complexity for standard API integrations.
Final Calculation: Factoring in a Processing Complexity Adjustment (PCA) of 1.00, the system totals **110 Gross Function Points (GFP)**.



Effort & Project Duration

- **Productivity Baseline:** Estimated development velocity is **10 function points per person-week**.
- **Total Estimated Effort:** 110 Function Points ÷ 10 = **11 person-weeks** of total work.
- **Team Capacity:** Project relies on **6 active team members**.
- **Estimated Duration:** 11 person-weeks ÷ 6 team members = **~1.83 weeks**.
- **Conclusion:** The project requires approximately **2 weeks of dedicated coding time** (excluding planning and documentation phases).



Detailed Project Phases

Requirements Analysis

01

Defining file types (PDF, CSV), user interactions, and security requirements.

System Design

02

Designing RAG pipeline, API architecture, and UI interaction flows.

Implementation

03

Core coding of document uploads, parsing, and LLM query handling.

Integration & Test

04

External API linking, unit testing, and edge case validation.

Deployment

05

Performance tuning, final security checks, and user guidelines.

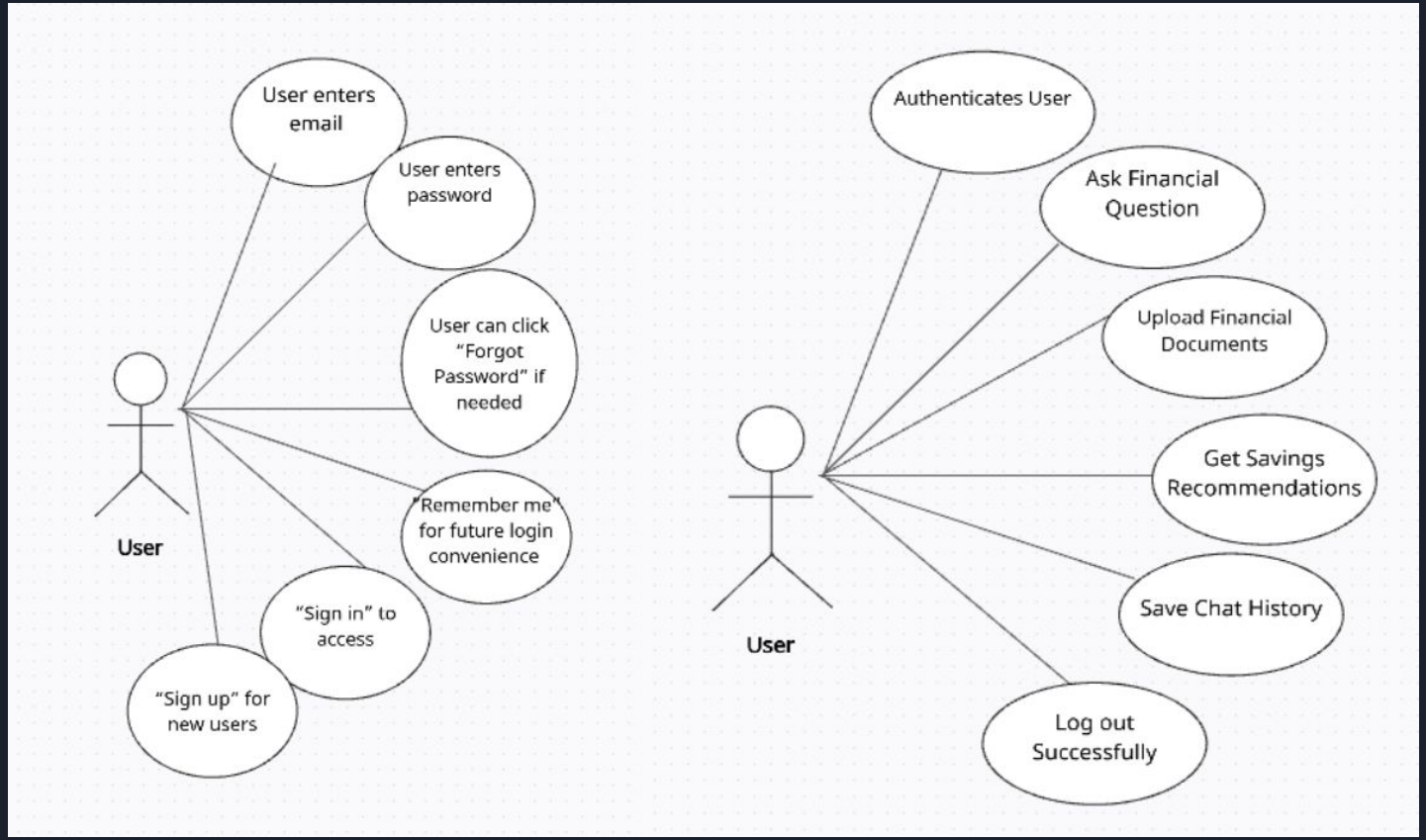
Non-functional Requirements

- Response times will be under 5 seconds for smaller inputs and under 1 minute for larger inputs
- Our system will support documents up to ~25 MB
- The user interface will be simple and easy to use
- The system shall protect user data and store it securely

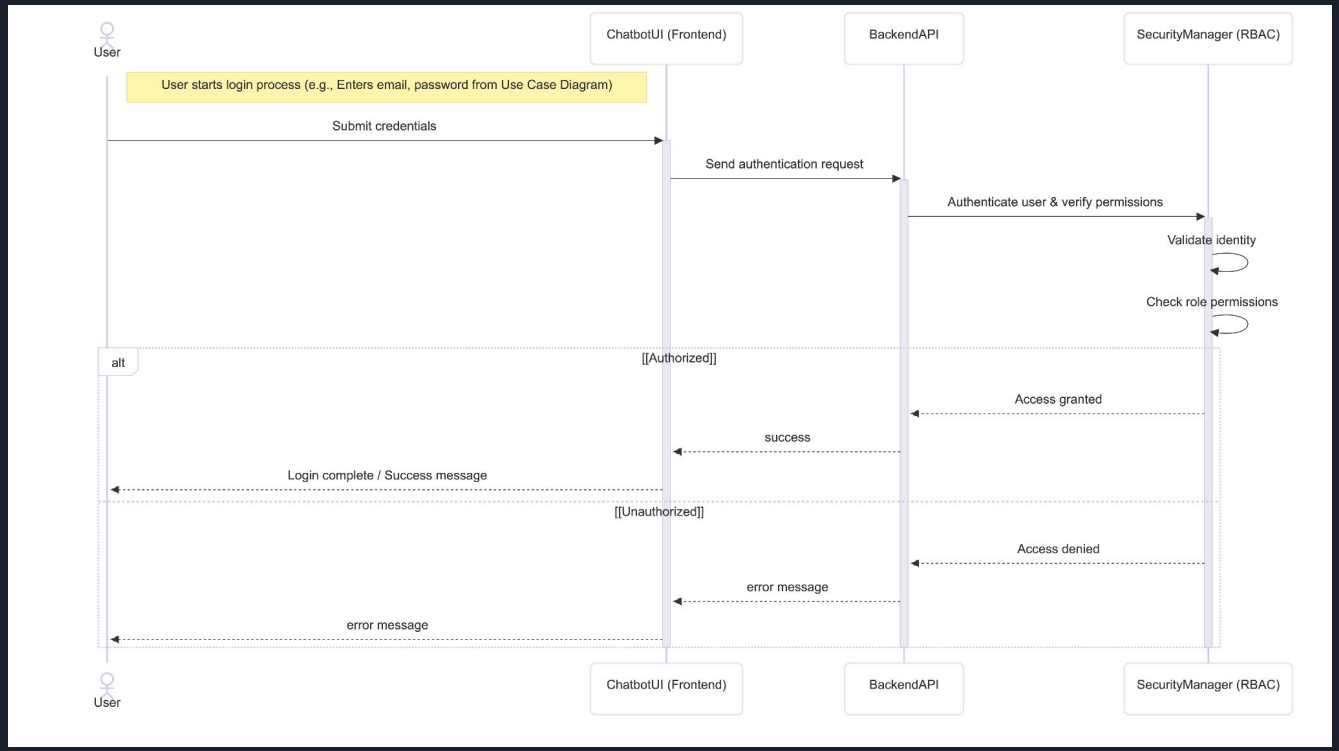
Functional Requirements

- The system shall accept PDF, CSV, TXT, & image uploads as context
- The system shall process documents and pass context to the LLM to support responses
- The system shall authenticate users using a login system
- The system shall display responses in the user interface

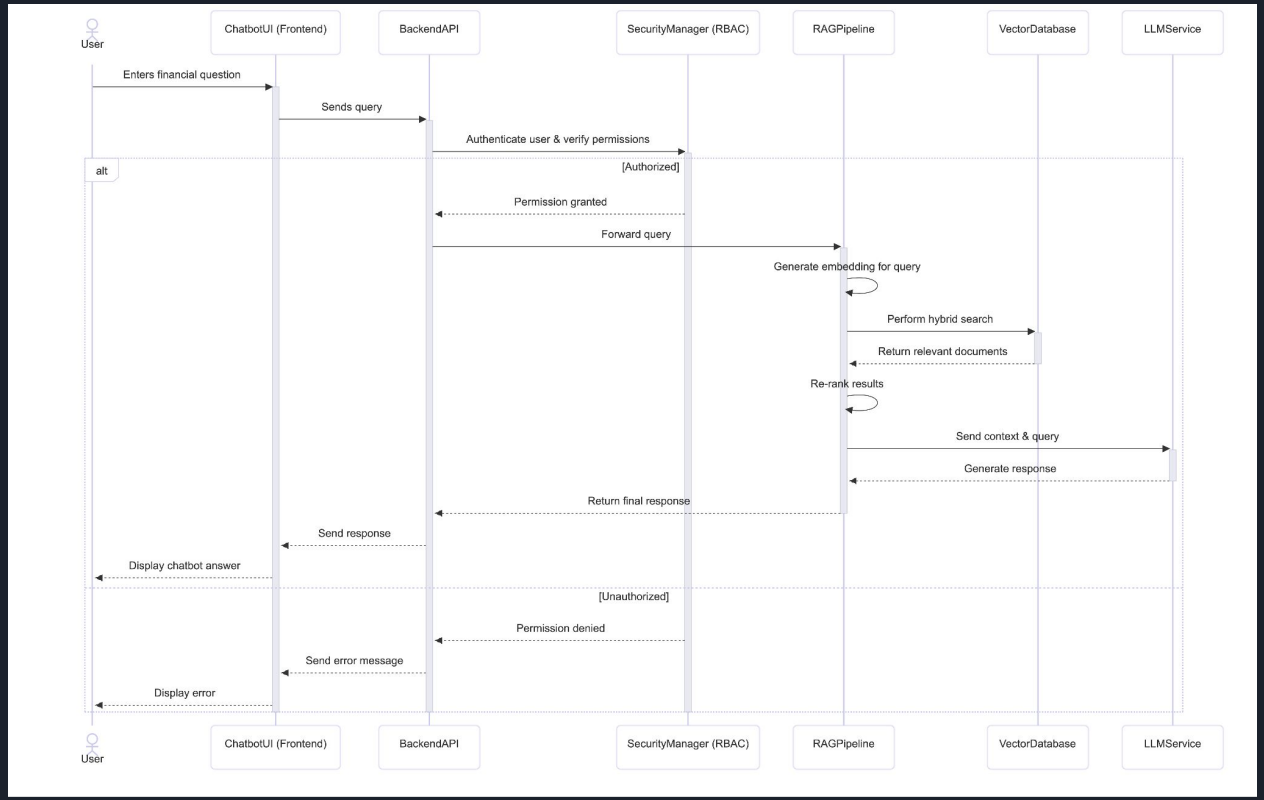
Use Case Diagrams (Login & Chatbot Interface)



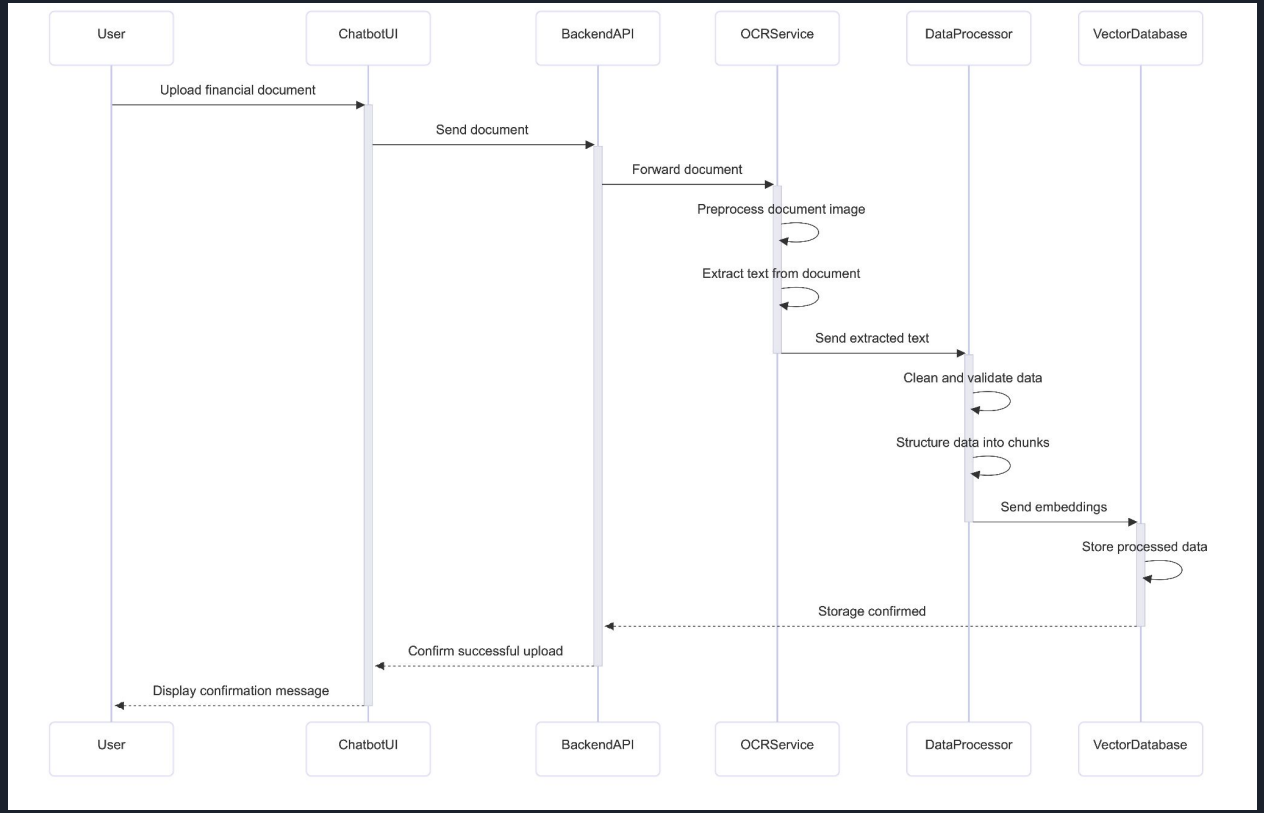
User Authentication



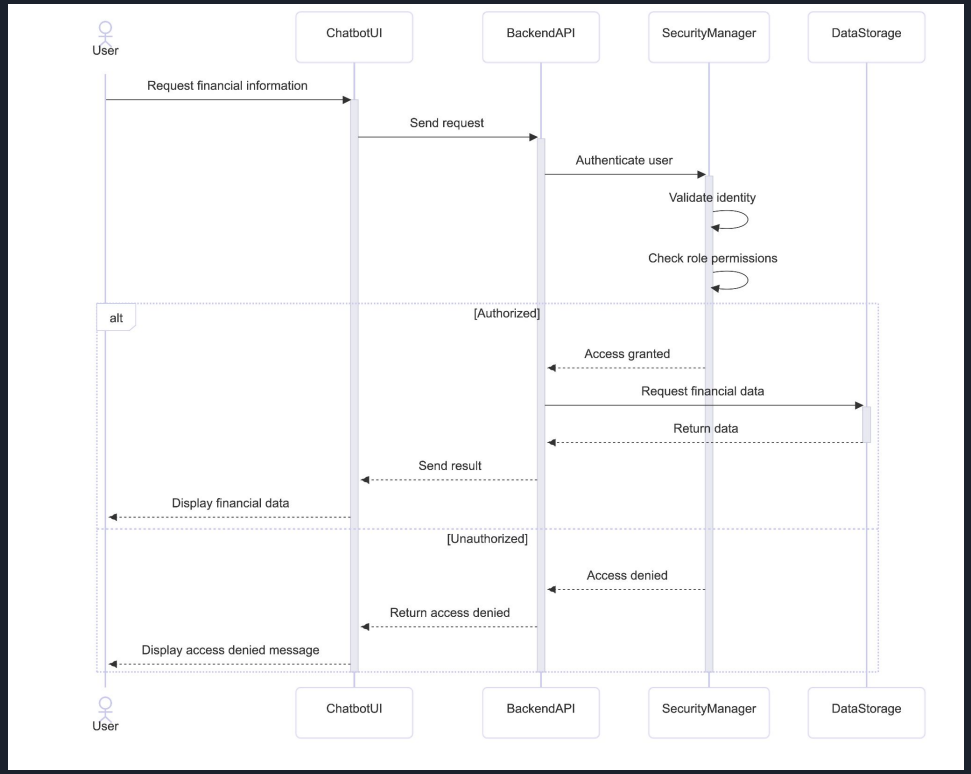
User Asks for Financial Question



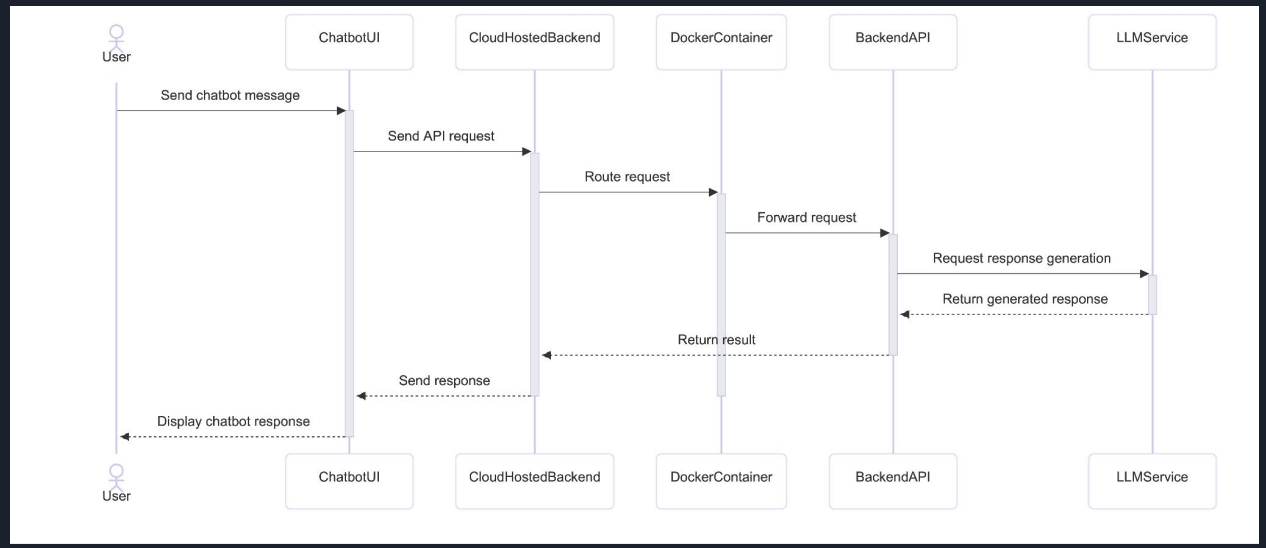
User uploads the financial document



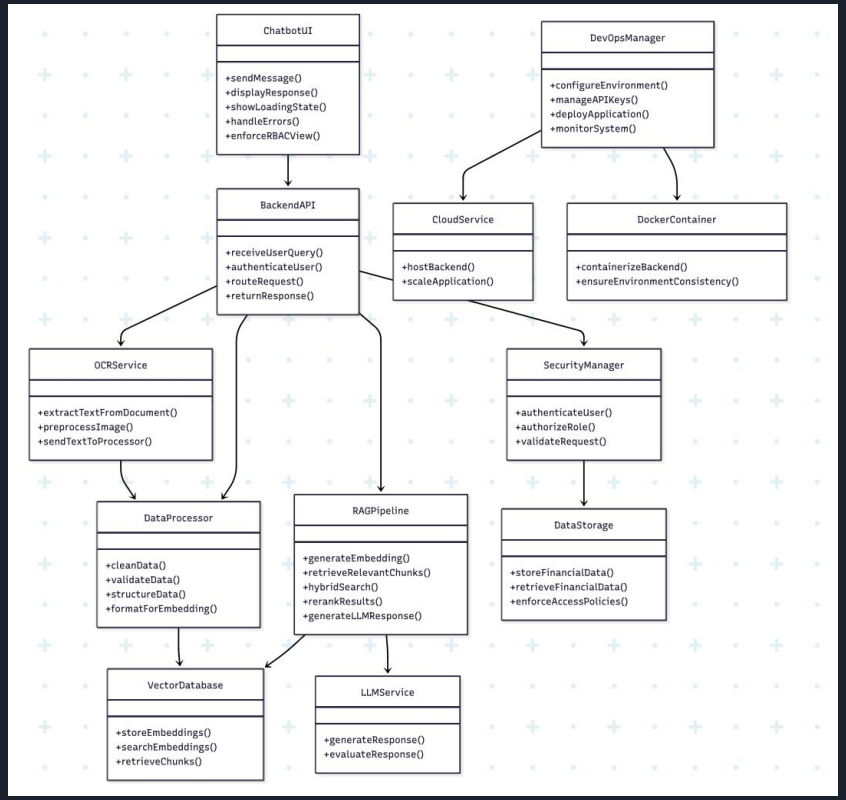
Secure Data with RBAC



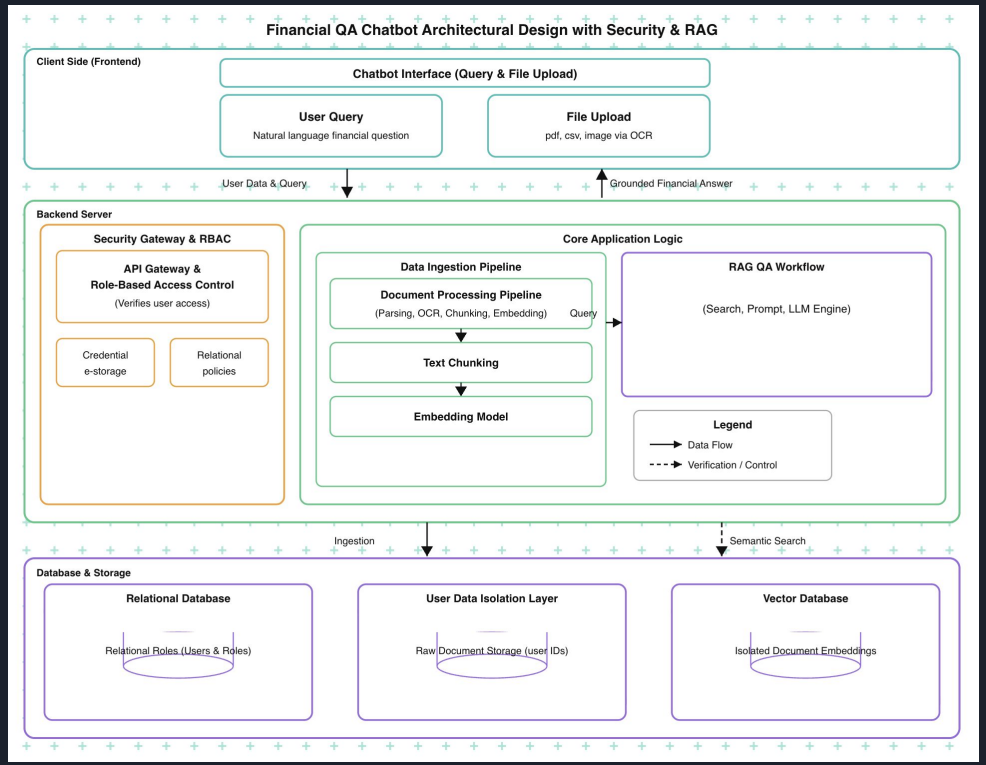
Chatbot Deployment and API Communication



Class Diagram



Architectural Design



Comparison with Systems

- LLMs (ChatGPT, Gemini, Claude, etc.)
  - LLMs are able to process documents and provide responses as well
  - LLMs have the capability to search the internet and pull additional context
  - Our system is fine tuned for the purpose of financial assistance
  - The data we use as context is not used for training LLMs [2]
- Financial Tools
  - Financial tools like excel can provide visuals and calculations
  - These tools require you to understand or fill in context
  - Our system is easier to use and simplifies responses
  - Our system does not require financial knowledge to use

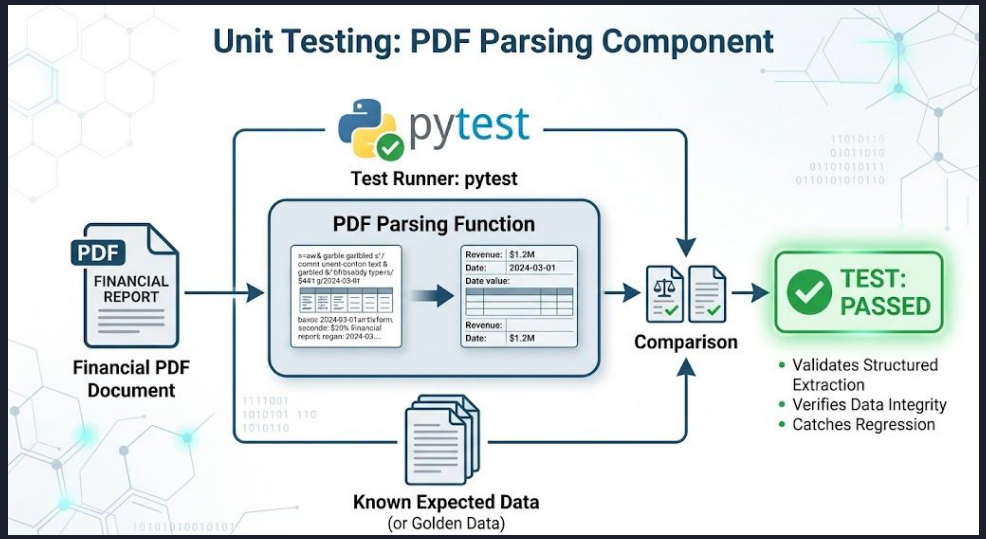
Testing and Validation

Importance of Testing and Validation

Unit Testing vs System Testing

PDF Parsing function

pytest framework



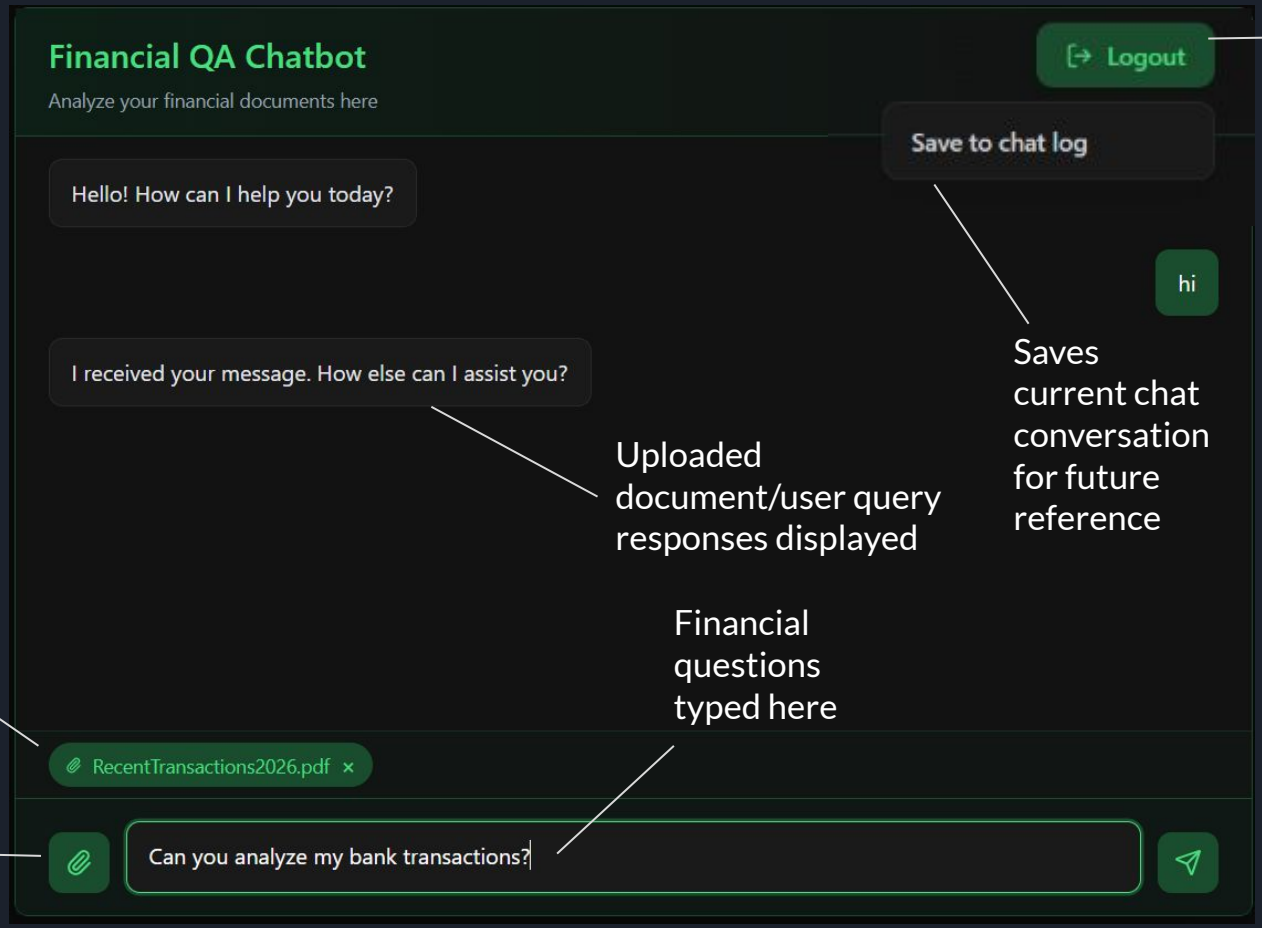
Passing Tests

Test Case	INPUT	EXPECTED RESULT	ACTUAL RESULT
Valid PDF Path	sample.pdf	Text successfully extracted from the PDF	PASSED
Invalid File Path	fake_file.pdf	Function raises FileNotFoundError	PASSED
Empty File Path	“”	Function raises ValueError	PASSED

```
PS C:\Users\Dev\Documents\GitHub\3354-team6> python -m pytest test_pdf_parser.py -v
========================================== test session starts ==========================================
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0 -- C:\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Dev\Documents\GitHub\3354-team6
collected 3 items

test_pdf_parser.py::test_valid_pdf PASSED                                                    [ 33%]
test_pdf_parser.py::test_invalid_path PASSED                                                 [ 66%]
test_pdf_parser.py::test_empty_path PASSED                                                   [100%]
```

Prototype of proposed application (Figma)



Ends current session, returns to login screen

Saves current chat conversation for future reference

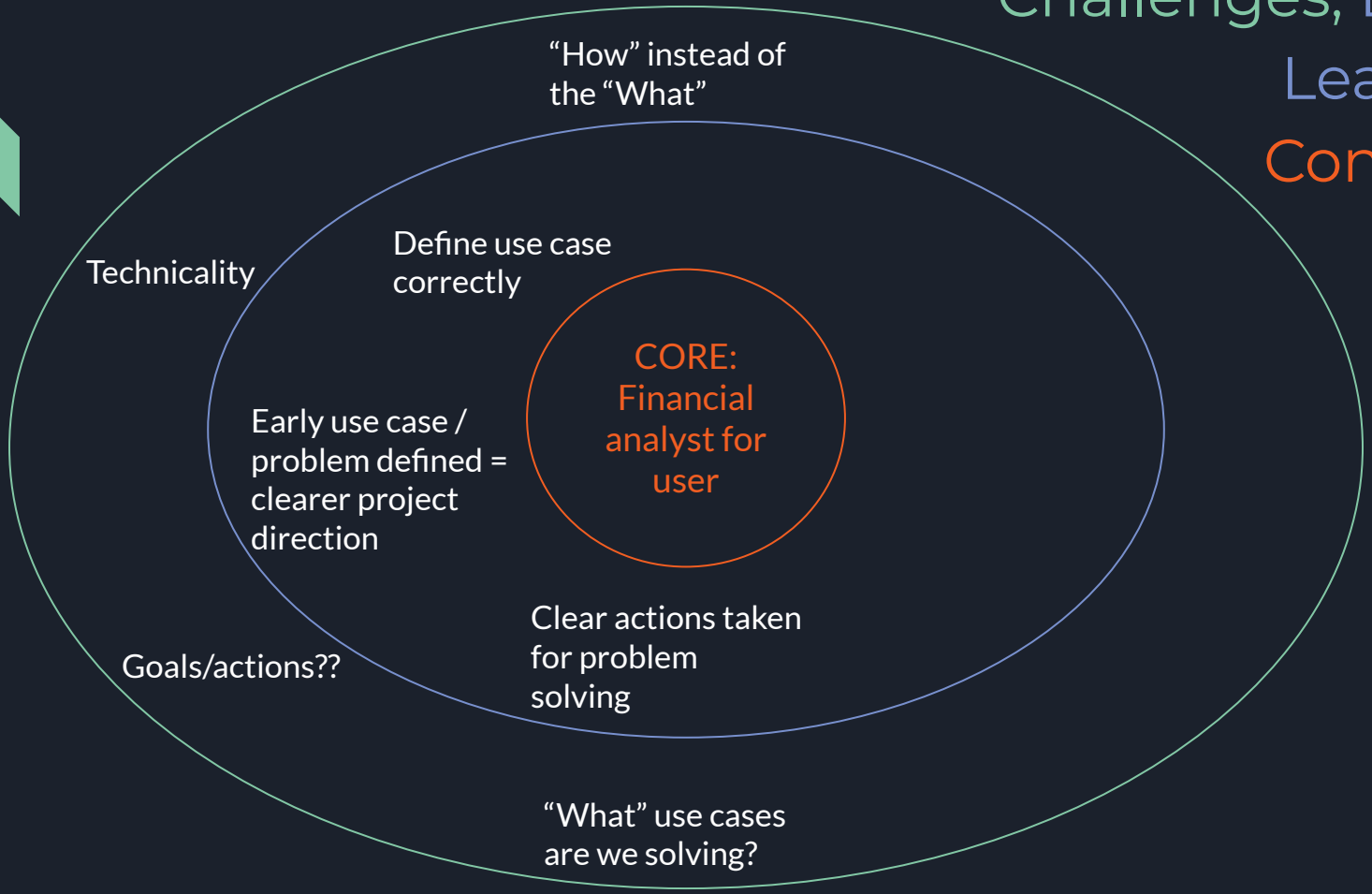
Uploaded document/user query responses displayed

Financial questions typed here

Uploaded financial document attached to current query, min 25 MB

Users can upload financial documents directly into chat

Challenges, Lessons Learned, & Conclusion



References

[1]
“Gemini Developer API pricing,” Google AI for Developers, 2025. https://ai.google.dev/gemini-api/docs/pricing#gemini-3.1-flash-lite-preview (accessed Mar. 29, 2026).
[2]
OpenAI, “Does OPEN AI API use API data for training?,” *OpenAI Developer Community*, Feb. 28, 2024. https://community.openai.com/t/does-open-ai-api-use-api-data-for-training/659053 (accessed Mar. 30, 2026).